Estándar de Codificación en Python

Curso propedéutico MCDI

A continuación se definen los estándares de codificación que se utilizarán en todos los cursos de INFOTEC que empleen Python como lenguaje de programación, con base en la guía de estilo oficial PEP 8 y en las buenas prácticas generales del lenguaje. Su cumplimiento es un criterio de evaluación en las actividades y prácticas de código.

Nombres de identificadores

- **Representativos:** los nombres de los identificadores deben ser representativos, brindando información sobre su contenido.
- **Variables y funciones/métodos:** notación snake_case (todo en minúsculas, palabras separadas por "_": nombre_descriptivo).
- **Constantes:** únicamente letras mayúsculas; si constan de varias palabras, separarlas con "_" (NOMBRE_DESCRIPTIVO).
- **Clases:** notación PascalCase (NombreDescriptivo) — ver sección "Nombres de clases".
- **Sin acentos ni "ñ":** los identificadores no deben contener acentos, diéresis, ni la letra "ñ", aunque el IDE los acepte.
- **Nombres de una sola letra:** deben evitarse, salvo convenciones aceptadas (p. ej., i, j, k para índices de ciclos, o x, y en contextos matemáticos).
- **Palabras reservadas:** no deben usarse como identificadores las palabras reservadas de Python (for, class, list, type, id, etc.), incluso si el intérprete lo permite en algunos casos.
- **Módulos y paquetes:** nombres cortos, en minúsculas, y sin guiones bajos si es posible (nombremodulo).

Estructura del código

- **Indentación:** 4 espacios por nivel; no deben mezclarse espacios y tabs.
- **Longitud de línea:** no debe exceder 79 caracteres (99 como máximo en casos justificados), conforme a PEP 8.
- **Espaciado:** un espacio alrededor de los operadores (=, ==, +, -, etc.) y después de cada coma.
- **Una instrucción por línea:** no debe usarse ";" para separar varias instrucciones en la misma línea.
- **Líneas en blanco:** dos líneas en blanco antes y después de cada definición de función o clase de nivel superior, y una línea en blanco entre métodos dentro de una clase.
- **Variables usadas:** el código solo debe incluir variables que efectivamente se utilicen.
- **f-strings:** debe preferirse f"..." para construir cadenas con valores interpolados, en lugar de concatenación con "+".
- **Comparación con None:** debe hacerse con "is" / "is not", nunca con "==" o "!=".

- **Imports explícitos:** deben evitarse los imports con comodín (from modulo import *); cada import debe ser explícito.
- **Manejo de recursos:** los recursos externos (archivos, conexiones, etc.) deben manejarse con la instrucción with, para asegurar su liberación correcta.

Comentarios y documentación

- **Comentarios útiles:** deben aportar valor para facilitar la comprensión del código; evitar comentarios obvios al leer la instrucción referenciada.
- **Encabezado de archivo:** las primeras líneas de cada archivo deben incluir, como mínimo:
  - Breve descripción del programa o módulo.
  - Una línea con el/los autor(es): nombre(s) y matrícula(s).
  - Una línea con la fecha de creación/modificación.
- **Docstrings:** cada función, método o clase debe incluir un docstring (""" """) inmediatamente después de su definición, antes de cualquier otra instrucción. El docstring de una función debe describir:
  - El propósito de la función.
  - Cada parámetro (Args).
  - El valor de retorno (Returns), si aplica.
- **Comentarios de una línea:** deben iniciar con "# " (numeral seguido de un espacio) e incluirse solo cuando aporten información no evidente en el código.
- **Sin código muerto:** no deben dejarse líneas de código sin uso comentadas; para eso está el control de versiones.

Nombres de clases

- **PascalCase y singular:** la primera letra de cada palabra debe ser mayúscula (NombreDescriptivo) y el nombre debe estar en singular.
- **Excepciones propias:** deben seguir también PascalCase y terminar con el sufijo "Error" cuando corresponda (por ejemplo, ValoresInvalidosError).

Funciones y métodos

- **Responsabilidad única:** cada función/método debe realizar una sola tarea y completarla en su totalidad.
- **Type hints:** las firmas de función deben incluir anotaciones de tipo para los parámetros y el valor de retorno, por ejemplo: def calcular_promedio(valores: list[float]) -> float:
- **Inicialización:** las variables deben definirse con un valor inicial antes de su primer uso.
- **Sin argumentos mutables por defecto:** evitar def f(lista=[]):; en su lugar, usar None y asignar el valor mutable dentro de la función.
- **Comprensiones o ciclos:** ambos son válidos. Usa la forma que te resulte más clara (una comprensión sencilla o un ciclo for explícito); elegir una u otra no afecta la calificación.

- **Sin efectos colaterales innecesarios:** evitar modificar variables globales o parámetros que no deberían cambiar; preferir funciones puras cuando sea posible.